```
        var error = new ProblemDetails                                          ❸
        {                                                                       ❸
            Title = "An error occurred",                                        ❸
            Detail = context.Exception.Message,                                 ❸
            Status = 500,                                                       ❸
            Type = " https://httpwg.org/specs/rfc9110.xhtml#status.500"         ❸
        };                                                                      ❸

        context.Result = new ObjectResult(error)                                ❹
        {                                                                       ❹
            StatusCode = 500                                                    ❹
        };                                                                      ❹
        context.ExceptionHandled = true;                                        ❺
    }
}
```

❶ ExceptionFilterAttribute is an abstract base class that implements IExceptionFilter.

❷ There's only a single method to override for IExceptionFilter.

❸ Building a problem details object to return in the response

❹ Creates an ObjectResult to serialize the ProblemDetails and to set the response status code

❺ Marks the exception as handled to prevent it propagating into the middleware pipeline

It's quite common to have an exception filter in your application if you are mixing API controllers and Razor Pages in your application, but they're not always necessary. If you can handle all the exceptions in your application with a single piece of middleware, ditch the exception filters and go with that instead.

You're almost done refactoring your `RecipeApiController`. You have one more filter type to add: result filters. Custom result filters tend to be relatively rare in the apps I've written, but they have their uses, as you'll see.

22.1.5 Result filters: Customizing action results before they execute